Chapter 9 – Case Studies (I): Business Problems

IERG4320/IEMS5726 Data Science in Practice | Cheat Sheet

Business Analytics Overview

商业分析：用数据驱动商业决策

BA vs DS: BA focuses on business decisions with structured data; DS uncovers patterns with any data using advanced algorithms.

BA偏商业决策; DS偏算法和模式发现

Importance: informed decisions, improved efficiency, increased revenue, reduced risk.

Recommender Systems

推荐系统：根据用户行为推荐物品（Amazon/Netflix/Spotify）

1st Gen: Basic Approaches

Collaborative Filtering 协同过滤：基于用户-物品交互

- **User-based**: find similar users, recommend their items
- **Item-based**: find similar items, recommend unwatched ones

```
from sklearn.metrics.pairwise import cosine_similarity
# User-based CF
user_sim = cosine_similarity(user_item)
sim_users = np.argsort(
  user_sim[user_id-1][::-1][1:11]
# Item-based CF
item_sim = cosine_similarity(user_item.T)
```

Content-based: recommend by item features (TF-IDF on genres).

基于内容：分析物品特征，推荐相似物品

Knowledge-based: rules/constraints (keywords, same company).

基于知识：用规则约束推荐

Limitations: cold start (new user/item), over-specialization.

冷启动问题：新用户/物品没有历史数据

2nd Gen: Hybrid & Matrix Factorization

Hybrid: combine multiple methods.

混合推荐：结合多种方法

- **Weighted**: $0.4 \times CF + 0.3 \times Content + 0.3 \times KB$
- **Switching**: CF for old users, content for new users
- **Cascade**: CF creates candidates, content re-ranks

Matrix Factorization: decompose $R \approx P \times Q^T$

矩阵分解：把用户-物品评分矩阵分解为两个低秩矩阵

R = rating matrix, P = user factors, Q = item factors.

$\hat{r}_{ui} = \mathbf{p}_u \cdot \mathbf{q}_i$, optimize via gradient descent.

```
def matrix_factorization(R, k, steps=5000,
                         alpha=0.0002, beta=0.02):
  P = np.random.normal(scale=1./k,size=(m,k))
  Q = np.random.normal(scale=1./k,size=(n,k))
  for step in range(steps):
    for u in range(m):
      for i in range(n):
        if R[u,i] > 0:
          error = R[u,i] - np.dot(P[u,:],Q[i,:])
          P[u,:] += alpha*(2*error*Q[i,:]-beta*P[u,:])
          Q[i,:] += alpha*(2*error*P[u,:]-beta*Q[i,:])
  return P, Q
# Or use sklearn TruncatedSVD
from sklearn.decomposition import TruncatedSVD
svd = TruncatedSVD(n_components=50)
user_factors = svd.fit_transform(user_item)
item_factors = svd.components_
```

3rd Gen: Deep Learning

DeepCoNN: two parallel CNNs on user/item reviews → embeddings → rating prediction.

DeepCoNN：用评论文本的CNN特征做推荐

Wide & Deep: Wide (linear, memorization) + Deep (NN, generalization). Joint training.

Wide&Deep：宽模型记忆特征交叉 + 深模型泛化新组合

```
class WideDeep(nn.Module):
  def __init__(self, n_users, n_movies, ...):
    self.wide = nn.Linear(20, wide_dim)
    self.user_emb = nn.Embedding(n_users,emb_sz)
    self.movie_emb = nn.Embedding(n_movies,emb_sz)
    self.deep = nn.Sequential(...)
    self.final = nn.Linear(wide_dim+1, 1)
```

GCN (Graph Convolutional Network):

图卷积网络：在图结构(社交网络)上做卷积，学习节点嵌入

Semi-supervised on graph data. Aggregate neighbor features.

```
from torch_geometric.nn import GCNConv
class GCN(torch.nn.Module):
  def __init__(self, in_ch, hid_ch, out_ch):
    self.conv1 = GCNConv(in_ch, hid_ch)
    self.conv2 = GCNConv(hid_ch, out_ch)
  def forward(self, x, edge_index):
    x = self.conv1(x, edge_index).relu()
    x = self.conv2(x, edge_index)
    return x
```

RecSys Evaluation: CTR, conversion rate, retention rate, NPS.

Stock Market Prediction

股票预测：用历史数据预测未来趋势（很难！）

Why difficult: black swans, market psychology, reflexivity, overfitting.

Data: yfinance API

```
import yfinance as yf
ticker = yf.Ticker("AAPL")
data = ticker.history(period="1y")  # OHLCV
# Multiple tickers
multi = yf.download(["AAPL","MSFT"],
  start="2024-01-01", end="2024-10-18")
# Company info
info = ticker.info  # name, sector, marketCap
# Financials
ticker.financials; ticker.balance_sheet
ticker.dividends; ticker.recommendations
```

OHLCV = Open/High/Low/Close/Volume

Time Series Data Split

- **Holdout**: train on past, test on future 简单划分
- **Rolling Window CV**: sliding window forward in time 滚动窗口交叉验证

EMA (Exponential Moving Average)

指数移动平均：近期价格权重更大

SMA = mean of last n prices; EMA = weighted, recent prices matter more.

Limitation: lagging indicator, assumes trend continues.

RNN & LSTM

RNN：循环神经网络，处理序列数据; LSTM：解决梯度消失

RNN: $h_t = f_W(h_{t-1}, x_t)$ 用上一时刻隐状态+当前输入

LSTM: adds gates to control information flow.

- **Forget gate**: what to discard from memory 遗忘门
- **Input gate**: what new info to store 输入门
- **Output gate**: what to output 输出门
- **Cell state**: long-term memory vector 细胞状态=长期记忆

```
# Simple PyTorch LSTM
rnn = nn.LSTM(10, 20, 2) # in=10,hid=20,layers=2
input = torch.randn(5, 3, 10) # seq,batch,feat
h0 = torch.randn(2, 3, 20)
c0 = torch.randn(2, 3, 20)
output, (hn, cn) = rnn(input, (h0, c0))
```

```
class StockLSTM(nn.Module):
  def __init__(self, input_size=5,
               hidden_size=50, num_layers=2):
    super().__init__()
    self.lstm = nn.LSTM(input_size, hidden_size,
      num_layers, batch_first=True, dropout=0.2)
    self.fc1 = nn.Linear(hidden_size, 25)
    self.fc2 = nn.Linear(25, 1)
    self.relu = nn.ReLU()
    self.dropout = nn.Dropout(0.2)
  def forward(self, x):
    out, _ = self.lstm(x)
    out = self.dropout(
      self.relu(self.fc1(out[:,-1,:])))
    return self.fc2(out)
```

输入：30天滑动窗口的OHLCV数据

Investment Strategies

投资策略：系统化的投资决策方案

Non-ML Strategies

- **Active**: try to beat market (growth/value investing)
- **Passive**: match market returns (index funds, buy&hold)
- **Value**: buy undervalued stocks (Warren Buffett)
- **DCA**: invest fixed amount at regular intervals 定投

ML Investment Strategies

1. **Directional**: predict up/down (binary classification)
   方向预测：涨=1，跌=0
2. **Return Prediction**: predict % gain over N days
   收益预测：预测未来N天回报率
3. **Volatility Prediction**: predict price fluctuation
   波动率预测：预测价格波动程度
4. **Portfolio Optimization**: optimal asset allocation
   组合优化：最优资产配置权重

Financial Metrics

金融评估指标

- **Total Return** $= \frac{\text{End}-\text{Begin}+\text{Div}}{\text{Begin}} \times 100\%$
- **Sharpe Ratio** $= \frac{R_p - R_f}{\sigma_p}$ 风险调整回报，越高越好
- **Max Drawdown** $= \frac{\text{Trough}-\text{Peak}}{\text{Peak}}$ 最大回撤
- **Sortino Ratio** $= \frac{R_p - R_t}{\text{Downside Dev}}$ 只看下行风险

DQN for Trading

深度Q网络：用神经网络替代Q表，处理大状态空间

RL for trading: Agent=trader, Env=market, State=prices+portfolio, Action=buy/sell/hold, Reward=profit.

```
class DQN(nn.Module):
  def __init__(self, state_size=31,
               action_size=3): # hold/buy/sell
    super().__init__()
    self.fc1 = nn.Linear(state_size, 64)
    self.fc2 = nn.Linear(64, 64)
    self.fc3 = nn.Linear(64, action_size)
  def forward(self, x):
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    return self.fc3(x)
```

Key components:

- **Replay Memory**: store (s,a,r,s') experiences 经验回放
- **Target Network**: separate network for stability 目标网络
- **ε-greedy**: explore→exploit, decay ε over time

Training loop: observe → act → reward → new state → update Q.

Key Models in Ch9:
RecSys: CF, Content, Hybrid, MF/SVD, DeepCoNN, Wide&Deep, GCN
Stock: EMA, LSTM (sliding window), DQN
Caveat: "If it really makes money, it won't be published." 能赚钱的模型不会发表的